

Unit 7: Query Processing & Optimization (10%)

Q1. Explain the steps involved in query processing.

ANS. Query Processing is converting a high-level SQL query into an efficient execution plan. Steps include:

1. **Parsing and Translation:**
 - Check SQL query for **syntax and semantic errors**.
 - Convert SQL query into **relational algebra** or intermediate form.
 - Example: SELECT name FROM Student WHERE marks > 70 → $\sigma_{\text{marks} > 70}(\text{Student})[\text{name}]$
2. **Query Optimization:**
 - DBMS generates **equivalent expressions** that produce the same result.
 - Applies **transformation rules** like:
 - **Selection pushdown:** Apply selection before joins to reduce data early.
 - **Join reordering:** Change the order of joins to reduce intermediate tuples.
 - Goal: Find a **low-cost execution plan**.
3. **Evaluation Plan / Physical Optimization:**
 - Decide **how** to execute each operation:
 - Which **join method** (nested loop, hash join, etc.)
 - Which **access path** (full scan, index scan)
 - Use **statistics** (number of tuples, size of table, index availability) to estimate **cost**.
4. **Query Execution:**
 - Execute the physical plan step by step.
 - Intermediate results may be stored temporarily in memory or disk.
 - Return the **final result to the user**.
5. **Optional: Materialized Views / Caching:**
 - If part of the query matches a **materialized view**, reuse results to **save computation**.

Key Points:

- Query processing ensures **correctness and efficiency**.
- Optimization reduces **I/O and CPU cost**, especially for large databases.
- Intermediate results are **temporary** and help in **breaking complex queries into smaller operations**.

Q2. What are the measures of query cost?

ANS. Query cost helps DBMS choose the **most efficient execution plan**. Main measures include:

1. **I/O Cost (Disk Access):**
 - Most expensive in DBMS.
 - Depends on **number of pages read/written**.
 - Example: Full table scan of 1,000 pages = 1,000 I/Os.
2. **CPU Cost:**
 - Time to **process tuples in memory** (selection, join, projection).
 - Example: Comparing 10,000 tuples for a join → CPU cycles matter.
3. **Memory / Buffer Usage:**

- Amount of **RAM required** for intermediate results or sorting.
- Example: Hash join may need memory for hash table; insufficient memory → disk spill → higher cost.
- 4. **Communication / Network Cost (Distributed DBMS):**
 - Cost of sending data across servers.
 - Important for distributed queries.
 - Example: Joining tables located on different servers.
- 5. **Number of Tuples Processed:**
 - More tuples → higher CPU and I/O cost.
 - Selection and projection reduce tuples early to **lower cost of later operations**.
- 6. **Plan Complexity:**
 - More complex operations (nested loops, multiple joins) → higher overall cost.
 - DBMS prefers **simpler plans if cost is similar**.
- 7. **Response Time vs Throughput:**
 - **Response time:** How fast a single query completes.
 - **Throughput:** How many queries DBMS can process in a given time.
 - Cost may be measured differently based on system goal.

Key Points:

- DBMS uses **statistics** (table size, index info, selectivity) to estimate costs.
- Goal of query optimizer: **minimize total cost** (I/O + CPU + memory).
- Cost estimation ensures **efficient query execution**, especially for large datasets.

Q3. Explain different methods to evaluate selection and join operations.**ANS. 1. Selection Operation (σ condition (R))**

- **Goal:** Select tuples that satisfy a condition.

Methods:

1. **Linear Search (Full Table Scan):**
 - Check **every tuple** in the relation.
 - Simple but slow for large tables.
2. **Using Index:**
 - Use **primary or secondary index** if available.
 - Only check tuples that **might satisfy the condition** → faster.
3. **Using Sorted Data:**
 - If data is sorted on the selection attribute, use **binary search**.

2. Join Operation ($R \bowtie S$)

- **Goal:** Combine tuples from two relations based on a condition.

Methods:

1. **Nested Loop Join:**
 - For each tuple in R, **check all tuples in S** for match.
 - Simple but **costly for large tables**.
2. **Indexed Nested Loop Join:**
 - If S has an **index** on join attribute, use it → reduces search.
3. **Sort-Merge Join:**
 - **Sort both relations** on join attribute, then merge.
 - Efficient for large **sorted datasets**.
4. **Hash Join:**
 - Build **hash table** on smaller relation using join attribute.
 - Scan the other relation and **check hash matches**.

- Very efficient for large unsorted tables.

Key Points:

- DBMS chooses join method based on **relation size, indexes, and memory**.
- **Selection pushdown:** Apply selection **early** to reduce tuples → cheaper join.

□ **Summary for Exam:**

- Steps: Parse → Translate → Optimize → Plan → Execute
- Query cost: I/O, CPU, memory, tuples processed
- Selection: Linear scan, index, sorted data
- Join: Nested loop, indexed loop, sort-merge, hash join

Q4. What is transformation of relational expressions? Explain with example.**ANS. Definition:**

- Transformation of relational expressions is the process of **rewriting a query (relational algebra expression) into an equivalent expression** that produces the **same result**, but can be executed more efficiently.
- Goal: Reduce **query execution cost** (less I/O, CPU, or memory usage).

Why Needed:

- Some queries may generate **large intermediate results** if executed directly.
- By transforming expressions, DBMS can **optimize the order of operations** (selection, projection, join) to reduce cost.

Common Transformation Rules:

1. **Selection Pushdown:**
 - Apply **selection early** to reduce tuples before joining.
 - $\sigma_{\text{condition}}(R \bowtie S) \rightarrow \sigma_{\text{condition}}(R) \bowtie S$
2. **Projection Pushdown:**
 - Apply **projection early** to reduce columns in intermediate results.
 - $\pi_{\text{columns}}(R \bowtie S) \rightarrow \pi_{\text{columns}}(\pi_{\text{needed_columns}}(R) \bowtie \pi_{\text{needed_columns}}(S))$
3. **Join Reordering:**
 - Change **order of joins** to minimize intermediate results.
 - $(R \bowtie S) \bowtie T \rightarrow R \bowtie (S \bowtie T)$, depending on table sizes and indexes.

Example:

SQL Query:

SELECT name

FROM Student S, Marks M

WHERE S.rollno = M.rollno AND M.marks > 70;

- Initial relational algebra:
 $\pi_{\text{name}}(\sigma_{\text{marks}>70}(\text{Student} \bowtie \text{Marks}))$
- Apply **selection pushdown**:
 $\pi_{\text{name}}(\text{Student} \bowtie \sigma_{\text{marks}>70}(\text{Marks}))$
- Now only **relevant tuples** from Marks are joined → **less computation**.

Key Points:

- Transformation helps in **query optimization**.
- Reduces **intermediate results, I/O cost, and CPU usage**.
- DBMS uses these transformations **automatically in query optimizer**.

Q5. Explain materialized views. What are their advantages?**ANS. Definition:**

- A **materialized view** is a **precomputed and stored result of a query**.
- Unlike regular views, which are **virtual** and computed on-the-fly, materialized views **store the query result physically** in the database.

✓ How it Works:

- Example:

```
CREATE MATERIALIZED VIEW TopStudents AS  
SELECT rollno, name, marks  
FROM Student JOIN Marks  
WHERE marks > 90;
```

- The result is stored as a **table**.
- When the query is executed later, DBMS can **read from the materialized view** instead of recomputing joins and selections.

Advantages:

1. **Faster Query Processing:**
 - Frequently used complex queries can be **answered quickly**.
2. **Reduces Computation:**
 - Joins, aggregations, and selection already **precomputed**.
3. **Supports Caching:**
 - Acts like a **cache** for expensive operations.
4. **Can Be Indexed:**
 - You can create **indexes on materialized views** to speed up queries further.

Disadvantages / Considerations:

- Needs **storage space**.
- Must **refresh** when base tables change (can be immediate, on demand, or periodic).

Example Use Case:

- E-commerce site:
 - Materialized view of **daily top-selling products**.
 - Avoids computing join and aggregation every time user requests the report.

Key Points:

- Materialized views **improve performance** for repeated queries.
 - They are **precomputed, stored, and sometimes indexed**.
 - Used extensively in **data warehousing and analytics**.
-

Unit 8: Database Security (5%)

Q1. What are the goals of database security?

ANS. Database security ensures that the database and its data are **protected from unauthorized access, misuse, or damage**. The main goals are:

1. **Confidentiality:**
 - Only authorized users can **read sensitive data**.
 - Example: Only HR can see employee salaries.
2. **Integrity:**
 - Prevent **unauthorized modification** of data.
 - Example: Only authorized users can update bank balances.
3. **Availability:**
 - Ensure the database is **accessible when needed**.
 - Example: System should remain operational even under attacks or failures.
4. **Authentication:**
 - Verify the **identity of users** before granting access.
 - Example: Using username/password, tokens, or biometric authentication.
5. **Authorization / Access Control:**
 - Grant **permissions** to users based on roles or rules.
 - Example: User A can **read** data, but only User B can **update** it.
6. **Auditing and Accountability:**
 - Keep track of **who did what and when**.
 - Helps in **detecting misuse or attacks**.

Key Points: Security ensures **confidentiality, integrity, availability, and accountability** of database data.

Q2. Explain access control model: DAC, MAC, RBAC.

ANS. Access control decides **who can do what** in a database. There are three main models:

1. DAC (Discretionary Access Control):

- **Owner of data** decides access rights.
- Permissions can be **granted or revoked** by the user.
- Example: In SQL, GRANT SELECT ON Employee TO User1;
- **Pros:** Flexible.
- **Cons:** Owner may give access to unauthorized users.

2. MAC (Mandatory Access Control):

- **System-enforced** rules based on **security labels**.
- Users cannot override rules.
- Example: Data labeled “**Top Secret**” → only users with “**Top Secret**” clearance can access.
- **Pros:** Strong security.
- **Cons:** Less flexible; complex to manage.

3. RBAC (Role-Based Access Control):

- Access is given **based on user roles**, not individual users.
- Example: Role “**Manager**” can **read and update** salaries; Role “**Employee**” can only **read**.

- **Pros:** Easy to manage for large organizations.
- **Cons:** Roles need to be carefully designed.

Key Points:

- **DAC:** User decides access
- **MAC:** System enforces strict access
- **RBAC:** Access based on roles

Q3. What is SQL injection? How can it be prevent?**ANS. Definition:**

- SQL Injection is a **security attack** where an attacker inserts **malicious SQL code** into user input to manipulate the database.

Example:

Input: ' OR '1'='1

Query: SELECT * FROM Users WHERE username = " OR '1'='1' AND password = ";

- This allows **unauthorized access** because condition '1'='1' is always true.

✓ Prevention Techniques:

1. **Input Validation:** Check and sanitize all user inputs.
2. **Parameterized Queries / Prepared Statements:**
 - Use placeholders instead of concatenating strings.
 - Example (Java JDBC):
 - PreparedStatement ps = con.prepareStatement("SELECT * FROM Users WHERE username=? AND password=?");
 - ps.setString(1, user); ps.setString(2, pass);
3. **Stored Procedures:** Avoid dynamic SQL in code.
4. **Least Privilege Principle:** Give **minimal access rights** to users and applications.
5. **Web Application Firewalls (WAF):** Detect and block suspicious queries.

Key Points:

- SQL Injection exploits **unvalidated user input**.
- Prevented by **validation, parameterized queries, and minimal privileges**.

□ Summary Table (Quick for Exams):

Topic	Key Points
Goals of Security	Confidentiality, Integrity, Availability, Authentication, Authorization, Auditing
DAC	User-controlled access
MAC	System-enforced access based on labels
RBAC	Access based on roles
SQL Injection	Malicious SQL input; prevent using validation, parameterized queries, least privilege

Unit 9: Indexing and Hashing (10%)

Q1. Explain B+ Tree Index File with Insertion and Deletion

ANS. Definition:

- A **B+ tree** is a **balanced tree index** used to efficiently store and search large amounts of data.
- All **data pointers (records)** are stored in **leaf nodes**, and internal nodes store **key values for navigation**.
- Leaf nodes are linked to allow **sequential access**.

✓ Properties:

1. All leaves are at the same level → **balanced tree**.
2. Internal nodes store **keys and child pointers**, not actual data.
3. Leaf nodes store **data pointers and keys**.
4. Each node has **minimum and maximum number of keys** based on order m .

✓ Insertion in B+ Tree:

1. Start at root and **find the correct leaf** for the new key.
2. Insert the key in **sorted order** in the leaf node.
3. If the leaf **overflows** (more than max keys):
 - **Split the leaf** into two nodes.
 - Promote the **middle key** to the parent node.
4. If the parent overflows → **recursively split** until root.

✓ Deletion in B+ Tree:

1. Locate the key in the leaf and **remove it**.
2. If leaf has **too few keys** (underflow):
 - **Borrow a key** from a neighboring sibling, or
 - **Merge** with sibling and update the parent.
3. Repeat recursively up to the root if needed.

✓ Advantages:

- Balanced → search, insertion, deletion **$O(\log n)$**
- Supports **range queries efficiently** because leaf nodes are linked.

✓ Example (Simplified):

- Insert keys 10, 20, 30 in leaf → if max 2 keys per leaf → split occurs.
- Leaf nodes: [10] → [20,30], middle key 20 promoted to root.

Q2. Differentiate between Static Hashing and Dynamic Hashing.

Feature	Static Hashing	Dynamic Hashing
Structure	Fixed number of buckets	Number of buckets can grow/shrink dynamically
Overflow	Overflow chains occur if bucket is full	Buckets can split dynamically when full → avoids long overflow chains
Flexibility	Less flexible for growing data	Flexible, adapts automatically to increasing/decreasing dataset

Feature	Static Hashing	Dynamic Hashing
Search/Insert/Delete	Simple and fast if few collisions	Slightly more complex due to bucket splitting, but still efficient
Space Utilization	Can be wasted due to fixed bucket allocation	More efficient storage utilization; buckets created as needed
Scalability	Poor for large or growing datasets	High; handles database growth efficiently
Performance under Load	Performance degrades if buckets overflow	Maintains good performance even with large datasets
Example	Simple hash function on student IDs	Linear hashing, extendible hashing
Maintenance	Easy to implement, no reorganization	Slightly complex due to bucket splits and directory updates

Key Point:

- **Static hashing** → fixed structure, simpler but may overflow.
- **Dynamic hashing** → adapts to database size, avoids overflow, better for large or growing datasets.

Q3. What are Bitmap Indices? Where are They Useful?**ANS. Definition:**

- A **bitmap index** uses a **bitmap (0/1)** to represent whether a **row satisfies a condition** for a given attribute.

✓ **How it works:**

- Suppose column Gender has values Male and Female.
- Bitmap for Male → 1 if row is male, 0 otherwise.
- Bitmap for Female → 1 if row is female, 0 otherwise.

✓ **Advantages / Uses:**

1. **Efficient for low-cardinality attributes** (few distinct values).
2. **Fast boolean queries** using bitwise operations (AND, OR).
3. **Less storage** than full index for small domains.
4. Common in **data warehouses and OLAP systems**.

✓ **Example:**

- Table of 6 rows, Gender column: M, F, F, M, M, F
- Bitmap index:
 - Male → 1 0 0 1 1 0
 - Female → 0 1 1 0 0 1

Q4. How is an Index Defined in SQL?**ANS. Definition:**

- In SQL, an **index** is a database object used to **speed up queries** by providing **quick access to rows** based on column values.

Syntax:

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

Example:

```
CREATE INDEX idx_student_roll  
ON Student(rollno);
```

- Now queries like `SELECT * FROM Student WHERE rollno = 101;`
 - Will use the index for **faster search**.

Notes:

- **Unique index:** Ensures **no duplicate values** in the indexed column.
- `CREATE UNIQUE INDEX idx_unique_roll`
- `ON Student(rollno);`
- **Composite index:** Index on **multiple columns** to improve queries involving multiple conditions.

Key Points:

- Indexes **improve query performance**.
 - Can be **B+-tree**, **hash**, or **bitmap** type.
 - May **increase storage** and **slow down inserts/updates** slightly.
-

Unit 10: PL/SQL Concepts (5%)

Q1. What is a Cursor in PL/SQL? Differentiate Between Implicit and Explicit Cursors

ANS. Definition:

- A **cursor** in PL/SQL is a **pointer to the result set of a query**.
- It allows **row-by-row processing** of SQL query results in PL/SQL blocks.

✓ Types of Cursors:

1. Implicit Cursor:

- Automatically created by PL/SQL **when a SELECT, INSERT, UPDATE, or DELETE statement is executed**.
- Used for **single-row queries**.
- Example:

```
DECLARE
  v_salary NUMBER;
BEGIN
  SELECT salary INTO v_salary FROM Employee WHERE emp_id = 101;
  DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;
```
- No explicit declaration; PL/SQL handles the cursor internally.

Features:

- Automatically opened and closed.
- Can check execution status using SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT.

2. Explicit Cursor:

- Declared **manually** for queries that **return multiple rows**.
- Allows **row-by-row processing**.

Syntax:

```
DECLARE
  CURSOR emp_cursor IS
    SELECT emp_id, name, salary FROM Employee;
  v_emp_id Employee.emp_id%TYPE;
  v_name Employee.name%TYPE;
  v_salary Employee.salary%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_emp_id, v_name, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_emp_id || ' ' || v_name || ' ' || v_salary);
  END LOOP;
  CLOSE emp_cursor;
END;
```

Key Points:

- Must **declare, open, fetch, and close** the cursor.
- Suitable for **processing multiple rows**.

Comparison Table: Implicit vs Explicit Cursors

Feature	Implicit Cursor	Explicit Cursor
Declaration	Not required	Required
Use	Single-row queries	Multi-row queries
Management	Automatic	Manual (OPEN, FETCH, CLOSE)
Control	Limited	More control (row-by-row processing)
Status Info	SQL%FOUND, SQL%NOTFOUND	Cursor%FOUND, Cursor%NOTFOUND

Q2. What is a Stored Procedure? Explain with Syntax and Example**ANS. Definition:**

- A **stored procedure** is a **named PL/SQL block stored in the database** that can be **executed repeatedly**.
- Helps **modularize code**, **reduce redundancy**, and **improve performance**.
- Can **accept parameters** (input, output, or both) to perform operations dynamically.
- Enhances **security** by restricting direct access to underlying tables; users can execute the procedure without needing full table privileges.
- Supports **transaction management** within the database using COMMIT or ROLLBACK.

Syntax:

```
CREATE OR REPLACE PROCEDURE procedure_name (parameters)
IS
BEGIN
    -- PL/SQL statements
END procedure_name;
```

Example:

```
CREATE OR REPLACE PROCEDURE IncreaseSalary(emp_id IN NUMBER,
increment IN NUMBER)
IS
BEGIN
    UPDATE Employee
    SET salary = salary + increment
    WHERE emp_id = emp_id;
    COMMIT;
END IncreaseSalary;
```

Execution:

```
BEGIN
    IncreaseSalary(101, 5000);
END;
```

Key Points:

- Can have **input, output, or input-output parameters**.
- Improves **reusability, maintainability, and security**.

Q3. What is a Database Trigger? Explain Different Types with Examples**ANS. Definition:**

- A **trigger** is a **PL/SQL block automatically executed** when a **specific event occurs** on a table or view.
- Used to **enforce business rules, audit changes, or maintain integrity**.

✓ Types of Triggers:**1. Before Trigger:**

- Executes **before an event** (INSERT, UPDATE, DELETE).
- Example:

```
CREATE OR REPLACE TRIGGER before_insert_employee
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    :NEW.hire_date := SYSDATE; -- Automatically set hire date
END;
```

2. After Trigger:

- Executes **after an event**.
- Example:

```
CREATE OR REPLACE TRIGGER after_insert_employee
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New employee added: ' ||
:NEW.name);
END;
```

3. Instead of Trigger:

- Used for **views** to perform DML operations.

4. Row-Level Trigger:

- Executes **once for each row** affected.
- Example above uses FOR EACH ROW.

5. Statement-Level Trigger:

- Executes **once per SQL statement**, regardless of how many rows are affected.

Key Points:

- Triggers are **automatic** and cannot be called explicitly.
 - Useful for **validation, auditing, and enforcing constraints**.
-